

Lab 1 - Core Terraform Advanced: Remote State, Locking and Workspaces

Full Lab Guide, Configuration & Source Code

Terraform / Ansible / Docker / Kubernetes Lab Series

Contents

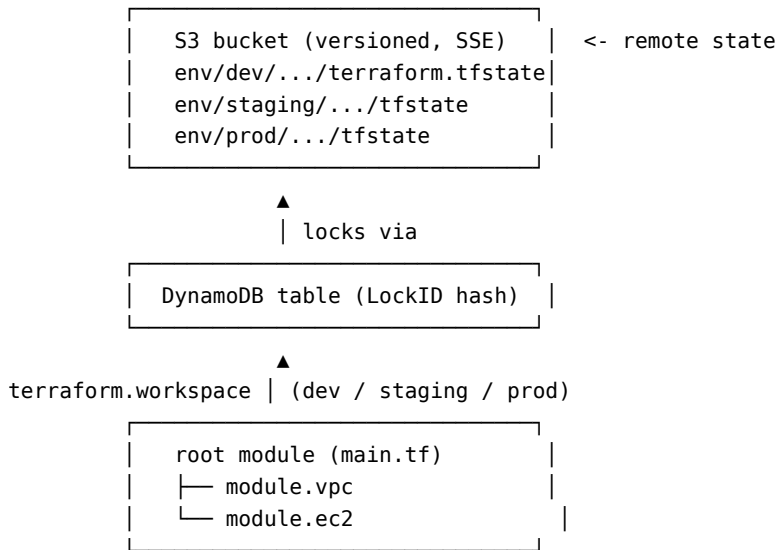
Use Case	1
Architecture	2
Directory Layout	2
Prerequisites	2
Step-by-Step Configuration & Execution	2
Step 1 — Bootstrap the backend (run once, local state)	2
Step 2 — Wire the backend into the root module	2
Step 3 — Initialize the root module	3
Step 4 — Create and select workspaces	3
Step 5 — Plan & apply per environment	3
Validation Steps	3
Cleanup	4
Key Concepts Reinforced	4
Appendix: Full Source Code	5
backend.tf	5
bootstrap/main.tf	5
main.tf	7
modules/ec2/main.tf	7
modules/ec2/outputs.tf	8
modules/ec2/variables.tf	8
modules/vpc/main.tf	9
modules/vpc/outputs.tf	10
modules/vpc/variables.tf	10
outputs.tf	10
variables.tf	10

Use Case

An organization needs to manage **dev, staging, and prod** AWS environments from a single Terraform codebase, without duplicating .tf files per environment, without losing state on a laptop crash, and without two engineers corrupting state by applying at the same time.

This lab teaches: - Remote state storage in **S3** (durable, versioned, encrypted) - State **locking** via **DynamoDB** (prevents concurrent-apply corruption) - **Terraform workspaces** to isolate dev/staging/prod state files - **Reusable modules** (vpc, ec2) driven by a per-environment config map - Safe bootstrapping of backend infrastructure (the classic “chicken-and-egg” problem)

Architecture



Directory Layout

```

lab1-core-remote-state-workspaces/
├── bootstrap/           # one-time: creates S3 bucket + DynamoDB lock table
│   └── main.tf
├── modules/
│   ├── vpc/            # VPC, subnets, IGW, route table
│   └── ec2/            # SG + EC2 instances, count driven by env
├── backend.tf          # remote backend config
├── main.tf             # root module, uses terraform.workspace
├── variables.tf        # per-environment sizing map
└── outputs.tf
  
```

Prerequisites

- Terraform `>= 1.5.0`
- AWS CLI configured (`aws configure`) with credentials that can create VPC/EC2/S3/DynamoDB
- An existing EC2 key pair in the target region (for `key_name`)

Step-by-Step Configuration & Execution

Step 1 — Bootstrap the backend (run once, local state)

```

cd bootstrap
terraform init
terraform apply -var="state_bucket_name=my-org-tfstate-lab1-<unique-suffix>"
  
```

Note the outputs `state_bucket` and `lock_table`.

Step 2 — Wire the backend into the root module

Edit `backend.tf` in the lab root and replace:

```
bucket = "REPLACE_WITH_YOUR_STATE_BUCKET"
```

with the bucket name from Step 1.

Step 3 — Initialize the root module

```
cd ..  
terraform init
```

Terraform will detect the S3 backend and initialize it.

Step 4 — Create and select workspaces

```
terraform workspace new dev  
terraform workspace new staging  
terraform workspace new prod  
terraform workspace select dev
```

Each workspace gets its own state file under `env/<workspace>/...` in S3 (because of `workspace_key_prefix = "env"` in `backend.tf`).

Step 5 — Plan & apply per environment

```
terraform plan -var="key_name=my-ec2-keypair" -out=dev.plan  
terraform apply dev.plan
```

Repeat for staging/prod after `terraform workspace select staging / prod`. Each workspace automatically picks up different `instance_type`, `instance_count`, and CIDR ranges from the `env_config` map in `variables.tf` via `local.cfg = var.env_config[terraform.workspace]`.

Validation Steps

1. Confirm remote state is being used (not local):

```
terraform state list  
aws s3 ls s3://<state_bucket>/env/dev/ --recursive
```

You should see `core-advanced-lab1/terraform.tfstate` under the `dev/` prefix.

2. Confirm state locking works: Open two terminals, run `terraform apply` in both simultaneously. The second one should fail with: `Error acquiring the state lock ... ConditionalCheckFailedException` Confirm the lock row appears while apply is running:

```
aws dynamodb scan --table-name terraform-state-lock
```

3. Confirm workspace isolation:

```
terraform workspace list  
terraform workspace select staging  
terraform state list      # different resource addresses/counts than dev
```

4. Confirm environment-specific sizing was applied:

```
terraform output instance_public_ips
```

dev → 1 IP, staging → 2 IPs, prod → 3 IPs (per env_config).

5. Reachability check:

```
for ip in $(terraform output -json instance_public_ips | jq -r '.[[]]'); do
  nc -zv -w3 $ip 22
done
```

Cleanup

```
terraform workspace select dev      && terraform destroy -var="key_name=my-ec2-keypair"
terraform workspace select staging  && terraform destroy -var="key_name=my-ec2-keypair"
terraform workspace select prod    && terraform destroy -var="key_name=my-ec2-keypair"
terraform workspace select default
terraform workspace delete dev staging prod
cd bootstrap && terraform destroy -var="state_bucket_name=my-org-tfstate-lab1-unique-suffix>"
```

Note: `prevent_destroy = true` on the S3 bucket in bootstrap will block destroy. Remove that lifecycle block deliberately before tearing down the bootstrap.

Key Concepts Reinforced

- Backend bootstrapping order-of-operations
- `terraform.workspace` for environment-aware logic
- DRY infra via parameterized modules
- State locking mechanics and how to prove they work

Appendix: Full Source Code

Every configuration and code file for this lab is listed below in full, organized by path, so the lab can be reproduced without needing the original repository.

backend.tf

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }

  # Remote backend: S3 for state storage, DynamoDB for locking.
  # Values here MUST match the bootstrap module outputs.
  # workspace_key_prefix isolates state per Terraform workspace (dev/stage/prod).
  backend "s3" {
    bucket          = "REPLACE_WITH_YOUR_STATE_BUCKET"
    key             = "core-advanced-lab1/terraform.tfstate"
    region          = "us-east-1"
    dynamodb_table = "terraform-state-lock"
    encrypt          = true
    workspace_key_prefix = "env"
  }
}

provider "aws" {
  region = var.aws_region
}
```

bootstrap/main.tf

```
# =====
# BOOTSTRAP: Creates the S3 bucket + DynamoDB table used as the
# Terraform remote backend (state storage + state locking).
# Run this ONCE, before the root module, with local state.
# =====

terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

```
variable "aws_region" {
  default = "us-east-1"
}

variable "state_bucket_name" {
  description = "Globally unique S3 bucket name for tf state"
  type        = string
}

resource "aws_s3_bucket" "tf_state" {
  bucket = var.state_bucket_name
  lifecycle {
    prevent_destroy = true
  }
}

resource "aws_s3_bucket_versioning" "tf_state" {
  bucket = aws_s3_bucket.tf_state.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "tf_state" {
  bucket = aws_s3_bucket.tf_state.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_s3_bucket_public_access_block" "tf_state" {
  bucket                = aws_s3_bucket.tf_state.id
  block_public_acls      = true
  block_public_policy    = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}

resource "aws_dynamodb_table" "tf_lock" {
  name           = "terraform-state-lock"
  billing_mode    = "PAY_PER_REQUEST"
  hash_key       = "LockID"
  attribute {
    name = "LockID"
    type = "S"
  }
}

output "state_bucket" {
  value = aws_s3_bucket.tf_state.bucket
}
```

```
output "lock_table" {
  value = aws_dynamodb_table.tf_lock.name
}
```

main.tf

```
locals {
  env      = terraform.workspace # dev / staging / prod
  cfg      = var.env_config[local.env]
  azs      = slice(["us-east-1a", "us-east-1b", "us-east-1c"], 0, length(local.cfg.public_subnets))
  common_tags = {
    Environment = local.env
    ManagedBy   = "Terraform"
    Lab         = "core-advanced-lab1"
  }
}
```

```
module "vpc" {
  source          = "./modules/vpc"
  env             = local.env
  vpc_cidr        = local.cfg.vpc_cidr
  public_subnet_cidrs = local.cfg.public_subnets
  azs             = local.azs
  tags            = local.common_tags
}
```

```
module "ec2" {
  source          = "./modules/ec2"
  env             = local.env
  vpc_id          = module.vpc.vpc_id
  subnet_ids      = module.vpc.public_subnet_ids
  instance_count  = local.cfg.instance_count
  instance_type   = local.cfg.instance_type
  key_name        = var.key_name
  tags            = local.common_tags
}
```

modules/ec2/main.tf

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical

  filter {
    name     = "name"
    values   = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

resource "aws_security_group" "web" {
  name_prefix = "${var.env}-web-sg-"
  vpc_id      = var.vpc_id

  ingress {
    description = "SSH"
  }
}
```

```
    from_port    = 22
    to_port      = 22
    protocol     = "tcp"
    cidr_blocks  = ["0.0.0.0/0"]
  }

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = merge(var.tags, { Name = "${var.env}-web-sg" })
}

resource "aws_instance" "web" {
  count                = var.instance_count
  ami                 = data.aws_ami.ubuntu.id
  instance_type       = var.instance_type
  subnet_id           = var.subnet_ids[count.index % length(var.subnet_ids)]
  vpc_security_group_ids = [aws_security_group.web.id]
  key_name            = var.key_name

  tags = merge(var.tags, { Name = "${var.env}-web-${count.index}" })
}
```

modules/ec2/outputs.tf

```
output "instance_ids" {
  value = aws_instance.web[*].id
}

output "public_ips" {
  value = aws_instance.web[*].public_ip
}
```

modules/ec2/variables.tf

```
variable "env" {
  type = string
}

variable "vpc_id" {
  type = string
}
```

```
variable "subnet_ids" {
  type = list(string)
}

variable "instance_count" {
  type = number
}

variable "instance_type" {
  type = string
}

variable "key_name" {
  type = string
}

variable "tags" {
  type    = map(string)
  default = {}
}
```

modules/vpc/main.tf

```
resource "aws_vpc" "this" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = merge(var.tags, { Name = "${var.env}-vpc" })
}

resource "aws_subnet" "public" {
  count                = length(var.public_subnet_cidrs)
  vpc_id               = aws_vpc.this.id
  cidr_block           = var.public_subnet_cidrs[count.index]
  availability_zone     = var.azs[count.index]
  map_public_ip_on_launch = true
  tags = merge(var.tags, { Name = "${var.env}-public-${count.index}" })
}

resource "aws_internet_gateway" "this" {
  vpc_id = aws_vpc.this.id
  tags   = merge(var.tags, { Name = "${var.env}-igw" })
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.this.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.this.id
  }
  tags = merge(var.tags, { Name = "${var.env}-public-rt" })
}

resource "aws_route_table_association" "public" {
  count = length(aws_subnet.public)
```

```
    subnet_id      = aws_subnet.public[count.index].id
    route_table_id = aws_route_table.public.id
  }
}
```

modules/vpc/outputs.tf

```
output "vpc_id" {
  value = aws_vpc.this.id
}

output "public_subnet_ids" {
  value = aws_subnet.public[*].id
}
```

modules/vpc/variables.tf

```
variable "env" {
  type = string
}

variable "vpc_cidr" {
  type = string
}

variable "public_subnet_cidrs" {
  type = list(string)
}

variable "azs" {
  type = list(string)
}

variable "tags" {
  type    = map(string)
  default = {}
}
```

outputs.tf

```
output "environment" {
  value = terraform.workspace
}

output "vpc_id" {
  value = module.vpc.vpc_id
}

output "instance_public_ips" {
  value = module.ec2.public_ips
}
```

variables.tf

```
variable "aws_region" {
```

```
    default = "us-east-1"
}

variable "key_name" {
    description = "Existing EC2 key pair name for SSH access"
    type        = string
}

# Per-workspace (per-environment) sizing map.
# terraform.workspace resolves to "dev", "staging" or "prod"
variable "env_config" {
    type = map(object({
        instance_type   = string
        instance_count  = number
        vpc_cidr        = string
        public_subnets = list(string)
    }))
    default = {
        dev = {
            instance_type   = "t3.micro"
            instance_count  = 1
            vpc_cidr        = "10.0.0.0/16"
            public_subnets = ["10.0.1.0/24"]
        }
        staging = {
            instance_type   = "t3.small"
            instance_count  = 2
            vpc_cidr        = "10.1.0.0/16"
            public_subnets = ["10.1.1.0/24", "10.1.2.0/24"]
        }
        prod = {
            instance_type   = "t3.medium"
            instance_count  = 3
            vpc_cidr        = "10.2.0.0/16"
            public_subnets = ["10.2.1.0/24", "10.2.2.0/24", "10.2.3.0/24"]
        }
    }
}
```